

Practica - 6 Estructuras de datos

Luis Gálvez

Tabla de contenidos

1 Proposito de la practica	1
1.1 Ejercicio 1 - Rutas de monitoreo (tuplas y listas)	1
1.1.1 Desarrollo	2
1.2 Ejercicio 2 - Metadatos y ecosistemas (diccionarios y conjuntos) .	3
1.2.1 Desarrollo	4
1.3 Preguntas	5
1.3.1 Desarrollo	5

1 Proposito de la practica

1.1 Ejercicio 1 - Rutas de monitoreo (tuplas y listas)

Contexto: Trabajas en un proyecto de conservación en la Sierra Nevada de Santa Marta. Los guardabosques realizan recorridos diarios y te envían las coordenadas de los puntos de avistamiento de fauna. Necesitas estructurar esta ruta temporal de forma segura, garantizando que las coordenadas individuales no se alteren, pero permitiendo que la ruta crezca a medida que reportan nuevos puntos.

Instrucciones de código:

1. Define tres variables (`punto_1`, `punto_2`, `punto_3`) que almacenen las coordenadas (Latitud, Longitud) como **Tuplas** estáticas (o su equivalente más seguro en tu lenguaje).
 - Punto 1: 11.1198, -74.0321
 - Punto 2: 11.1250, -74.0280
 - Punto 3: 11.1302, -74.0215
2. Crea una colección secuencial y mutable (**Lista** en Python/Julia o **Vector/Lista** en R) llamada `ruta_avistamiento` que contenga los tres puntos iniciales en orden.
3. Los guardabosques acaban de reportar un cuarto punto (11.1355, -74.0150). Utiliza el método nativo de tu lenguaje para **agregar** esta nueva coordenada al final de tu ruta.

4. Utilizando la indexación dinámica (indexación negativa o comandos como `length/end`), extrae el **último punto** de la ruta y guárdalo en una variable llamada `ultimo_reporte`.
5. Crea una nueva lista/vector paralela llamada `elevaciones` que contenga las alturas de los cuatro puntos: `[850.5, 920.0, 1050.2, 1180.8]`.
6. Calcula e imprime la elevación máxima alcanzada en el recorrido y el número total de puntos visitados (vértices), utilizando las funciones nativas (ej. `max()`, `len()`, `length()`).

1.1.1 Desarrollo

```
# Registro de ruta de avistamiento

#-----
# Definir los puntos como tuplas (latitud, longitud)
punto_1 = (11.1198, -74.0321)
punto_2 = (11.1250, -74.0280)
punto_3 = (11.1302, -74.0215)

#-----
# Crear una lista mutable con los puntos iniciales
ruta_avistamiento = [punto_1, punto_2, punto_3]

#-----
# Agregar un nuevo punto reportado
nuevo_punto = (11.1355, -74.0150)
ruta_avistamiento.append(nuevo_punto)

#-----
# Extraer el último punto utilizando indexación negativa
ultimo_reporte = ruta_avistamiento[-1]

#-----
# Crear lista paralela de elevaciones
elevaciones = [850.5, 920.0, 1050.2, 1180.8]

#-----
# Calcular estadísticas del recorrido
elevacion_maxima = max(elevaciones)
total_puntos = len(ruta_avistamiento)

#-----
# Imprimir resultados
print("Ruta de avistamiento:")
print(ruta_avistamiento)
```

```

print("\nÚltimo reporte registrado:")
print(ultimo_reporte)

print("\nElevación máxima alcanzada:")
print(elevacion_maxima, "msnm")

print("\nNúmero total de puntos visitados:")
print(total_puntos)

```

Ruta de avistamiento:

```
[(11.1198, -74.0321), (11.125, -74.028), (11.1302, -74.0215), (11.1355, -74.015)]
```

Último reporte registrado:

```
(11.1355, -74.015)
```

Elevación máxima alcanzada:

```
1180.8 msnm
```

Número total de puntos visitados:

```
4
```

1.2 Ejercicio 2 - Metadatos y ecosistemas (diccionarios y conjuntos)

Contexto: Estás diseñando el esquema de base de datos para los Parques Nacionales Naturales (PNN). Necesitas estructurar los atributos de un parque usando un modelo de clave-valor (similar a GeoJSON) y depurar una lista de observaciones de campo que tiene datos duplicados.

Instrucciones de código:

1. Crea un **Diccionario** (o Lista nombrada en R) llamado `pnn_tayrona` que contenga exactamente las siguientes claves y valores:
 - `"nombre"`: "PNN Tayrona"
 - `"area_hectareas"`: 15000
 - `"abierto_turismo"`: Verdadero (tipo Booleano)
 - `"ecosistemas"`: Una lista/vector con los textos "Manglar", "Bosque Seco", y "Coral".
2. Emplea la función o método de acceso seguro (ej. `.get()` en Python o condicionales en R/Julia) para intentar extraer la clave `"fecha_creacion"`. Como no existe, debe devolver el texto "Dato no disponible" sin que el programa colapse. Imprime el resultado.
3. Actualiza el diccionario añadiendo una nueva clave llamada `"departamento"` con el valor "Magdalena".
4. Recibes un reporte de campo crudo con los ecosistemas observados hoy, el cual contiene duplicados: `observaciones_crudas = ["Manglar",`

"Coral", "Bosque Seco", "Manglar", "Coral", "Matorral"]
Convierte esta colección en un **Conjunto** (*Set* o usando `unique()`) para eliminar los duplicados y guárdalo en la variable `observaciones_unicas`.

5. Utiliza una **operación lógica de conjuntos** (intersección) entre la lista oficial de ecosistemas del parque (guardada dentro de tu diccionario en el paso 1) y tus `observaciones_unicas`. Imprime el resultado para saber qué ecosistemas oficiales fueron efectivamente avistados hoy.

1.2.1 Desarrollo

```
# Información del PNN Tayrona

# Crear el diccionario con la información del parque
pnn_tayrona = {
    "nombre": "PNN Tayrona",
    "area_hectareas": 15000,
    "abierto_turismo": True,
    "ecosistemas": ["Manglar", "Bosque Seco", "Coral"]
}

#-----
# Acceso seguro a una clave inexistente
fecha_creacion = pnn_tayrona.get("fecha_creacion", "Dato no disponible")

print("Fecha de creación:")
print(fecha_creacion)

#-----
# Agregar una nueva clave al diccionario
pnn_tayrona["departamento"] = "Magdalena"

#-----
# Reporte de observaciones con duplicados
observaciones_crudas = [
    "Manglar",
    "Coral",
    "Bosque Seco",
    "Manglar",
    "Coral",
    "Matorral"
]

#-----
# Convertir a conjunto para eliminar duplicados
observaciones_unicas = set(observaciones_crudas)
```

```

print("\nObservaciones únicas:")
print(observaciones_unicas)

#-----
# Intersección entre ecosistemas oficiales y observados
ecosistemas_oficiales = set(pnn_tayrona["ecosistemas"])

ecosistemas_avistados = ecosistemas_oficiales.intersection(observaciones_unicas)

print("\nEcosistemas oficiales avistados hoy:")
print(ecosistemas_avistados)

```

Fecha de creación:

Dato no disponible

Observaciones únicas:

```
{'Coral', 'Bosque Seco', 'Matorral', 'Manglar'}
```

Ecosistemas oficiales avistados hoy:

```
{'Coral', 'Manglar', 'Bosque Seco'}
```

1.3 Preguntas

1. **Sobre el Ejercicio 1:** En términos de topología espacial y seguridad del código, ¿cuál es la ventaja de guardar las coordenadas individuales (X, Y) como *Tuplas* inmutables, pero guardar la ruta completa como una *Lista* mutable? ¿Qué pasaría si intentas modificar directamente la latitud del `punto_1` usando `punto_1[0] = 12.0000` en Python o Julia?
2. **Sobre el Ejercicio 2:** En R, existe un debate constante entre usar `c()` o `list()`. Si estuvieras programando en R y necesitaras almacenar la lista de "ecosistemas" dentro de las propiedades del parque, ¿qué comando usarías para agrupar esos textos específicos y por qué?
3. **Pregunta General (Slicing):** Explica brevemente la diferencia crítica en el comportamiento de extracción de rangos (*slicing*) si ejecutas `ruta[1:3]` en **Python** comparado con ejecutar el mismo código en **R** o **Julia**. ¿Cuántos elementos te devuelve cada uno y por qué?

1.3.1 Desarrollo

1. De esta manera las coordenadas del punto se mantienen fijas, sin embargo es posible modificar la lista de puntos en caso de que se desee modificar alguno. Si se intentase alterar los valores de `punto_1` arrojaría un error de tipo, ya que la tupla es por definición inmutable.
2. En R, `c()` permite agrupar elementos del mismo tipo, mientras que `list()`

puede agrupar elementos de distintos tipos, por ejemplo strings, booleanos, numericos y otros. Para el caso planteado en particular, basta con usar `c()`, debido a que la lista de ecosistemas estará compuesta en su totalidad por strings.

3. Existen 2 diferencias clave. La primera es que en python los indices empiezan desde 0, por lo que si la función lo llama por el indice 1 realmente estaría llamando el segundo elemento, mientras que en R y en Julia se llamaría el primer elemento. La segunda diferencia clave es que en python, no se toma en cuenta el ultimo indice del slice, es decir que en la expresión `ruta[1:3]` no se tomaría en cuenta el tercer elemento, mientras que en R y en Julia si se contempla el ultimo elemento de la expresión o el “techo”, es decir que devuelven 1 elemento más que python.